

The previous chapter introduced some of the basic concepts of linked lists using *unordered linked lists*. Those concepts form the basis for this chapter and many of the chapters that follow. In particular, this chapter will examine *ordered linked lists* and includes the development of a linked list class which utilizes the `Node` class as shown in Fig. 5.1 in the previous chapter. The `insert` routine will be developed first, followed by the `delete` routine, and a driver main program utilizing the `LinkedList` class.

6.1 Creating the Insert Method

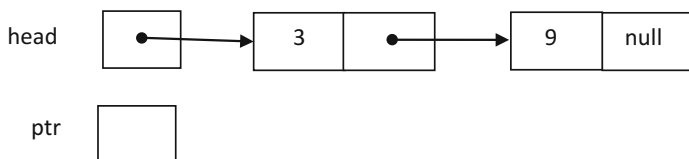
This section will explore the development of an `insert` method. Although in the last chapter the test program in Sect. 5.5 inserted nodes, they were restricted to being inserted only at the beginning of the list and it was done from the main program instead of from a method. Here, a method will be developed as part of a `LinkedList` class that will be capable of inserting nodes in the proper location at the beginning of a list, middle of the list, the end of a list, or in an empty list. Further, it will not allow for duplicate items in the list, but this is left as an exercise at the end of the chapter. Since this method will need to be sent the `item` to be inserted it will have one parameter. Also, since it should be able to inform the invoking program whether the item was inserted or not, it will return a `boolean` value.

6.1.1 Inserting in the Middle of the List

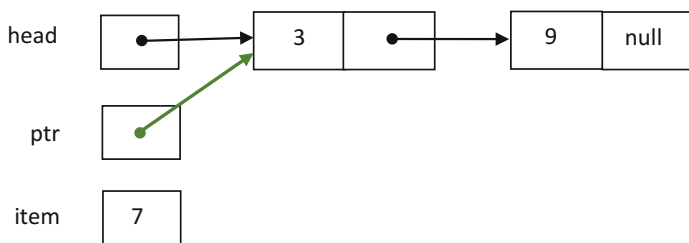
When developing code utilizing references, it is tempting to begin with the first item on a list. Unfortunately this can sometimes lead to algorithms that work well for a specialized case such as inserting an item at the beginning of the list but can lead to

code that is not as efficient for cases such as inserting items in the middle of the list, or other special cases such as at the end of the list or in an empty list. Instead, it is often helpful to create the algorithm for the more general case that might occur more frequently, such as inserting an item somewhere in the middle of the list, and then work out the special cases including the beginning and end of the list, or an empty list.

Given this advice, consider the existence of the ordered list below. Note that `head` references the first item in the list, and that the list is an ordered linked list which means that each node in the list follows one another in a prescribed order, and in this case it is in ascending order. Other names for `head` can be `first`, `list`, and `front`, but this text will chose `head` for the beginning of the list and if necessary use `tail` for the end of the list. Here, `ptr` is used as a shortened name for pointer and although technically the arrows are references, it is convenient to use `ptr` or `p` as is done in many other languages.



At first it might be frustrating to assume the existence of a list and not know how it was created in the first place. However, again when trying to create an algorithm, it can be easier to start with a generic case first. Assuming the above list, what if one wanted to insert the number 7? First, the list would need to be searched to determine where the item should be inserted. In order to search the list, a loop can be employed. The variable `ptr` would need to be initialized to the same value as `head` as shown below:



While `ptr` is not equal to `null` the loop can continue to iterate and each time through the loop the value of `ptr` would need to be updated to reference the next node in the list by taking on the value of the `next` field in the node referenced by

`ptr`. The basic loop structure to accomplish the above should look familiar and similar to the loop that was used in the iterative method to output the contents of a linked list presented in Sect. 5.6.

```
ptr = head;
while(ptr != null) {
    //body of loop
    ptr = ptr.getNext();
}
```

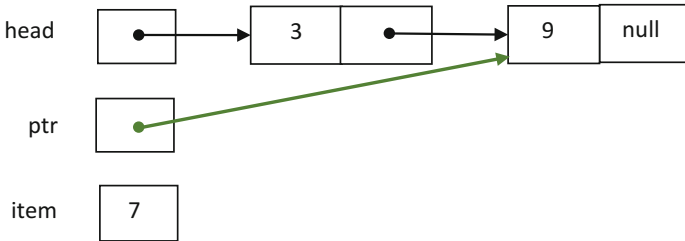
In the case where the list is empty, `head` is `null` as will be `ptr`, so the loop will not iterate as discussed later. Should the list not be empty, the loop would continue to iterate until `ptr` takes on the value of `null` from the last node in the list. However, is there any need to continue on to the end of the linked list? As was the case with array-based lists, when an item being searched for is found, there is no need to continue searching whether the linked list is ordered or unordered. However, if an item is not in the list then the answer would be yes in an unordered list since all the items would need to be searched. The answer is no for an ordered list since once `ptr` has gone beyond where the item should be located, there is no need to continue searching the list. This is an advantage of ordered linked lists over unordered lists and this can be included in the code as follows:

```
ptr = head;
while(ptr != null && ptr.getData() < item) {
    //body of loop
    ptr = ptr.getNext();
}
```

While the data in the current node is less than the item being searched for, the loop can continue to iterate. The test for `null` at the end of the list should be included for the cases where all the items in the list are less than the item being searched and `ptr` eventually would take on the value of `null`. Note that the order of operations in the conditional part of the `while` loop is critical. Should the order of the operands be reversed, it is possible that `ptr` could take on the value of `null`, and when `ptr.getData()` is invoked there could be a `null` pointer exception. By keeping the order above, should `ptr` be `null`, the first part of the `&&` statement would be `false`, and the second part of the `&&` would not be executed thus avoiding the execution error. Recall from a first course or text in programming such as *Guide to Java* [2] that this is known as a short circuit.

In looking at the previous diagram, it should seem obvious to the reader that the number 7 should be inserted immediately after the node containing the 3. However,

this cannot be determined by the algorithm since the contents of the next node have not been inspected. For all the algorithm knows, the subsequent node could contain a 4, not a 9, and the number 7 should not be inserted. Instead, the `ptr` needs to reference the next node to see what number is in it. So, since `ptr` is not equal to `null` and the number in the current node referenced by `ptr` is less than the value 7 in `item`, `ptr` is updated to reference the next node in the linked list as shown below:



The `ptr` variable now references the node containing the 9, and it can be checked to determine that the loop should quit iterating since 9 is not less than the 7 in `item`. A new node should be created containing the 7 and it should be inserted after the node containing the 3. Unfortunately, `ptr` is no longer referencing the node containing the 3. Could `head` be used? Yes, but only in this particular instance. What if there were many more nodes prior to the node containing the 9? The result is that `head` is not a viable choice due to the number of nodes that could exist prior to the location for insertion. Instead a second variable is needed to reference the previous node which will be called `prev` in this text. Initially when `ptr` references the first node in the list, there is no previous node, so `prev` is initialized to `null` as shown in Fig. 6.1.

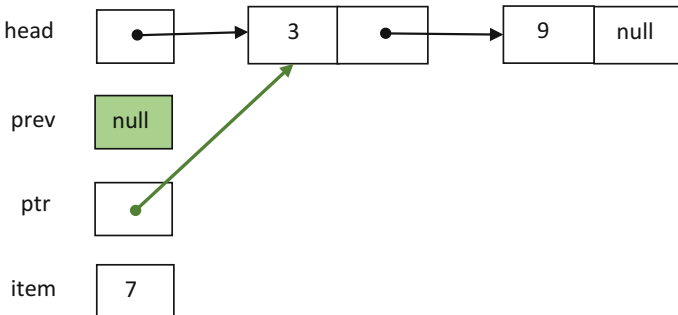
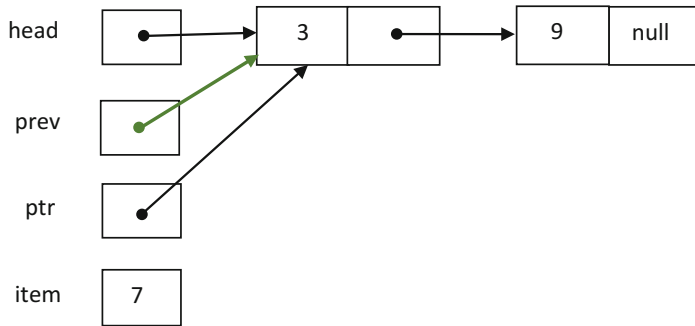
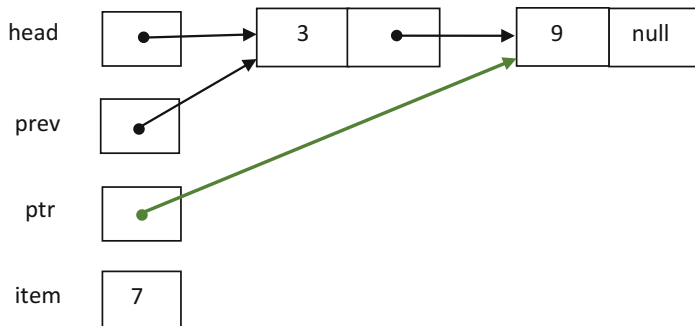


Fig. 6.1 Linked list with initial values for `prev` and `ptr`

Then during each iteration, `prev` needs to be set to the current node prior to moving `ptr` to the next node in the list. This is accomplished by copying the reference in `ptr` to `prev` as shown below:



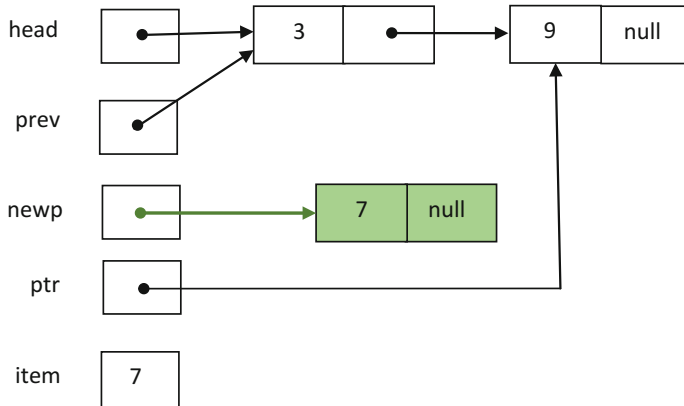
Then `ptr` can be move to reference the next node in the list as before.



The updated code to accomplish the above is shown below:

```
ptr = head;
prev = null;
while(ptr != null && ptr.getData() < item) {
    prev = ptr;
    ptr = ptr.getNext();
}
```

Now a new node can be created to include the 7 after the 3 referenced by `prev` and prior to the 9 referenced by `ptr`. A variable of type `Node` called `newp`, which stands for new pointer, is used. Then a new node needs to be created with the next field of the node set to null by the constructor in the `Node` class, and the data field of the node needs to be set to 7 as shown below:



The code to accomplish this is as follows:

```
Node newp = new Node();
newp.setData(item);
```

Or alternatively, the following can be used:

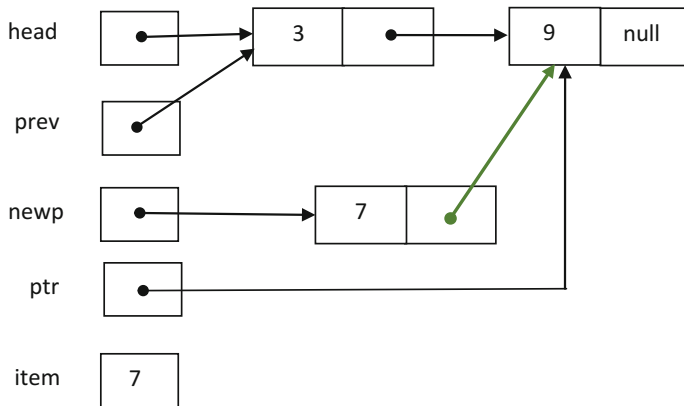
```
Node newp = new Node(item);
```

The new node then needs to be inserted into the linked list, and there are two references that need to be created: one from the node containing the 3 to the new node containing the 7, and one from the node containing the 7 to the node containing the 9. But which reference should be created first? In one sense it doesn't matter, but a general rule of thumb, or heuristic, is to create new links before removing old links. The reason for this is that on occasion by removing an old link, a link needed to get to a particular node might be lost. In this particular case, it might be better to create a new reference from the node containing the 7 before changing the reference in the node that contains the 3.

To create this new reference, the `next` portion of the node containing the 3 needs to reference the node containing the 9 and there are multiple ways this can be accomplished. However, another heuristic is to generally take the shortest route. In this case it is easier to use the reference from the variable `ptr` as illustrated in the following line of code,

```
newp.setNext(ptr);
```

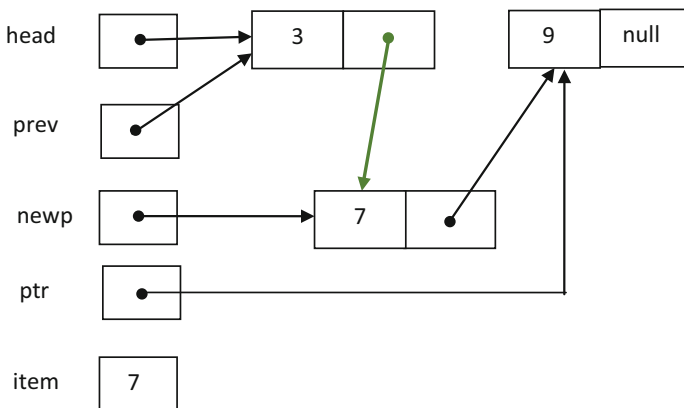
and the following drawing:



Then after the new node has been linked to the remaining part of the linked list, the connection from the node containing the 3 that references the node containing the 9 can be changed to refer to the new node containing the 7. However, which node should be used to access the `next` section of the node containing the 3: `head` or `prev`? Again, there could be many nodes between the first node referenced by `head` and the one referenced by `prev`. This illustrates that one must be careful when writing code since there are often multiple links that could be used. Instead of choosing the one that seems convenient for the particular case at hand, a programmer must consider the other situations and choose the one that works in many of the other possibilities that might exist. So, it is better to use the reference from `prev` rather than `head` as shown in the following instruction:

```
prev.setNext(newp);
```

and the following figure:



Putting the four lines of code together results in the following segment:

```
Node newp = new Node();
newp.setData(item);
newp.setNext(ptr);
prev.setNext(newp);
```

But what if the item is already in the linked list? The while loop as written stops when the data in the current node referenced by `ptr` is less than the number in `item`. As mentioned at the beginning of this chapter, duplicates will not be allowed in this linked list. What should be done is to check the `data` field of the current node that `ptr` is referencing to see if `item` is equal to it. If it is not, then the item should be inserted as shown in the following modified code segment:

```
if(ptr.getData() != item) {
    Node newp = new Node();
    newp.setData(item);
    newp.setNext(ptr);
    prev.setNext(newp);
}
```

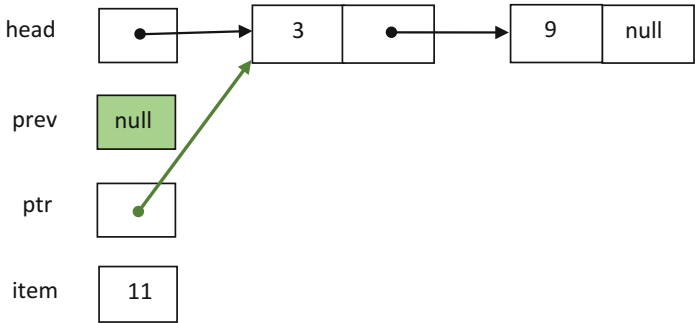
Combining the above code with the previous while loop results in the following code:

```
ptr = head;
prev = null;
while(ptr != null && ptr.getData() < item) {
    prev = ptr;
    ptr = ptr.getNext();
}
if(ptr.getData() != item) {
    Node newp = new Node();
    newp.setData(item);
    newp.setNext(ptr);
    prev.setNext(newp);
}
```

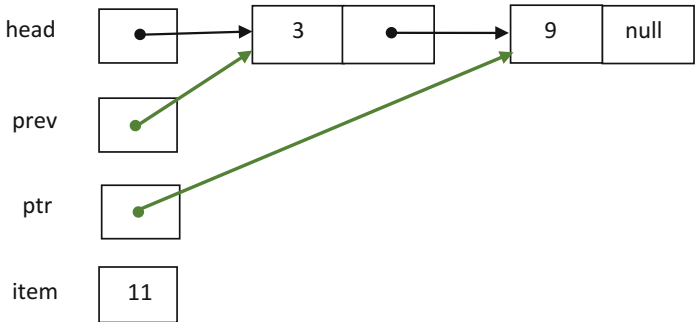
6.1.2 Inserting at the End of the List

Having written an algorithm for inserting an item in the middle of a linked list, how can an item be inserted at the end of the list? Although it is tempting to instead look at inserting an item at the beginning of the list, the insertion of a node at the

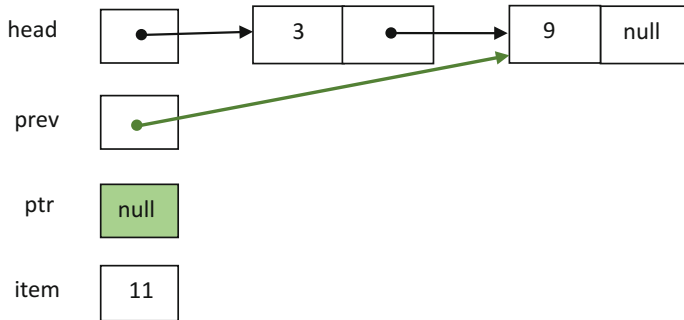
beginning requires that `head` be altered whereas inserting a node at the end does not. Since inserting at the end shares some similarities with inserting in the middle, it is helpful to look at this situation next. Using the above code segment one can walk through the code to see if it works for this scenario. Starting with the previous initial list in Fig. 6.1, what if one wanted to insert an 11? Again, the variable `ptr` would be initialized with the same reference as `head` and `prev` would be initialized to `null` as follows:



Since `ptr` is not equal to `null` and 3 is less than the 11 in `item`, the value in `prev` would reference the same node as `ptr` and then `ptr` would reference the next node in the linked list as shown below:



Again, `ptr` would not be equal to `null`, the 9 in the node referenced by `ptr` is less than the 11 in `item`, and another iteration of the loop occurs. The `prev` variable would then reference the same node as `ptr` and since there are no more nodes in the list, `ptr` would not point to a subsequent node, but rather take on a value of `null` from the next portion of the last node in the linked list as follows:



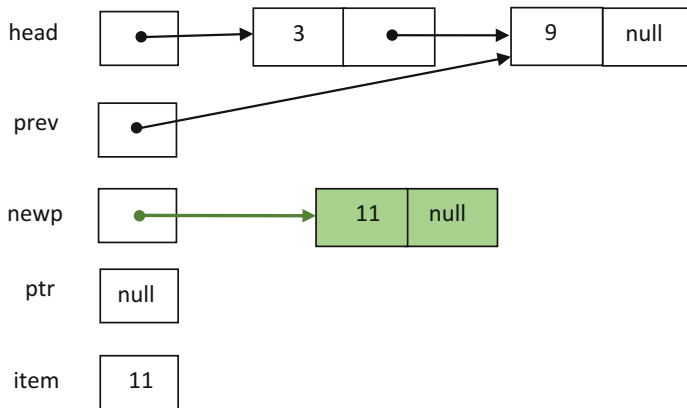
As the loop iterates back to the condition in the while loop, the value in `ptr` is now `null` so the loop stops iterating and continues on to the following `if` statement. But there is a problem with this `if` statement that is similar to the previous problem with the `while` statement. Since `ptr` is `null`, then `ptr.getData()` will cause an exception. So a check to see whether `ptr == null` needs to be added to the `if` statement and it should be prior to the `ptr.getData()` to avoid a short circuit as before. However, should the logical relation be `&&` or `||`? In the previous `while` statement it was an `&&` because both conditions needed to be `true` prior to moving on to the next node in the list. Here the situation is different because if `ptr` is equal to `null`, it is an indication that the end of the list has been reached, and `item` needs to be inserted. The result is that if this condition is `true`, there is no need to execute `ptr.getData()` and the then section of the `if` statement needs to be executed, so `||` is the proper choice.

```

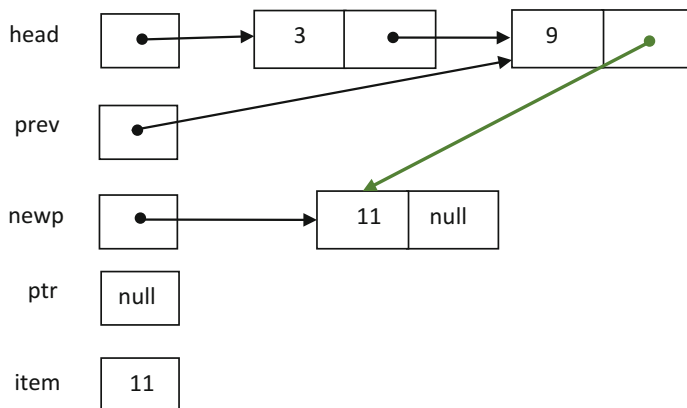
ptr = head;
prev = null;
while(ptr != null && ptr.getData() < item) {
    prev = ptr;
    ptr = ptr.getNext();
}
if(ptr == null || ptr.getData() != item) {
    Node newp = new Node();
    newp.setData(item);
    newp.setNext(ptr);
    prev.setNext(newp);
}

```

Again, it is imperative that `ptr == null` come first to avoid an exception. Since `ptr` is `null`, the code in the then section of the `if` statement is executed and a new node is created as follows:



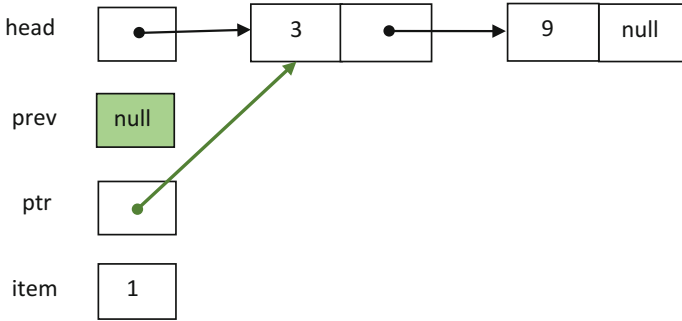
The next section of the new node is set to the same value as `ptr`. Even though the next section of the new node is already set to `null` by the constructor, since `ptr` is `null`, the next section is also `null` indicating the new end of the linked list. Lastly, the next section of the node referenced by `prev` is updated to reference the new node as follows:



Thus the number 11 is added to its proper place at the end of the linked list. What is interesting to notice is that the only change needed to the code is the `ptr == null` in the `if` statement to accommodate inserting at the end of the list. This is the reason why working on the more generic case first helps when implementing specific cases such as this.

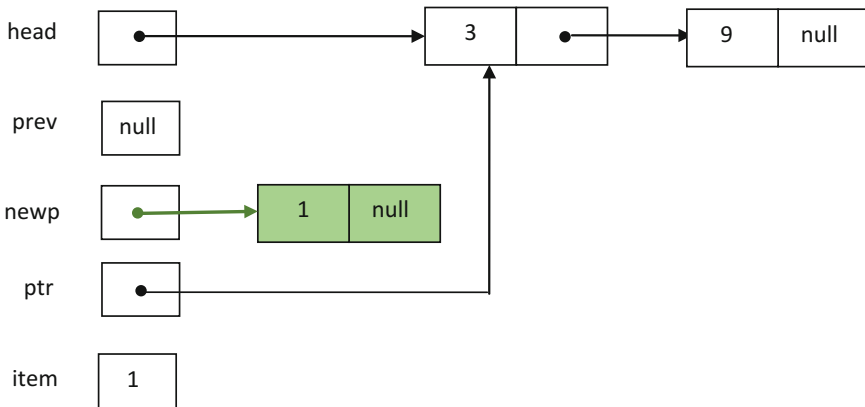
6.1.3 Inserting at the Beginning of a List

Having developed code to insert in the middle and end of a list, it is time to turn to the task of inserting at the beginning of a list. As alluded to in the previous subsection, inserting at the beginning of the list requires altering head, which will require some alterations to the previous code segment. Again using the initial drawing in Fig. 6.1, what if a 1 were to be inserted at the beginning of the list? As shown below, ptr is referencing the first node in the list, and prev has been set to null.

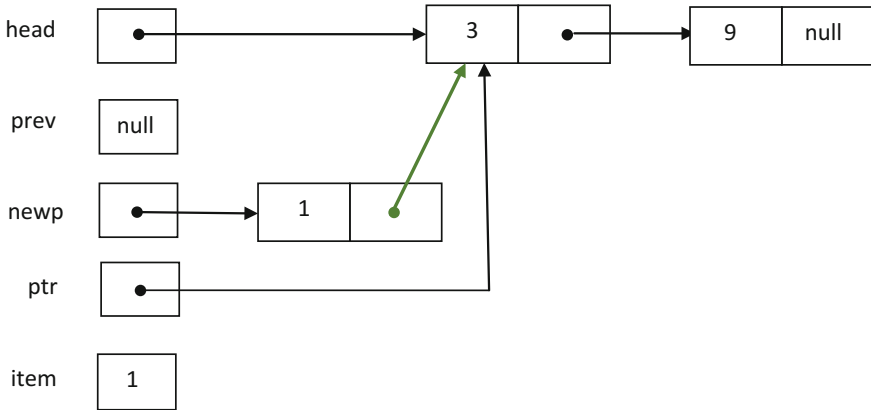


Upon reaching the while statement, ptr is not equal to null, but the 3 in the node referenced by ptr is not less than the 1 in item. The result is that the loop does not iterate because the correct spot in the list has been found. The execution of the segment jumps to the if statement where ptr is not equal to null, so the second part of the condition is checked where 1 is not equal to 3 indicating that it is not a duplicate, and execution can proceed into the then section of the if statement.

As before, a new node is created that contains the number 1 as shown below, where the linked list has been stretched out to make room for the new node at the beginning of the list as shown below:



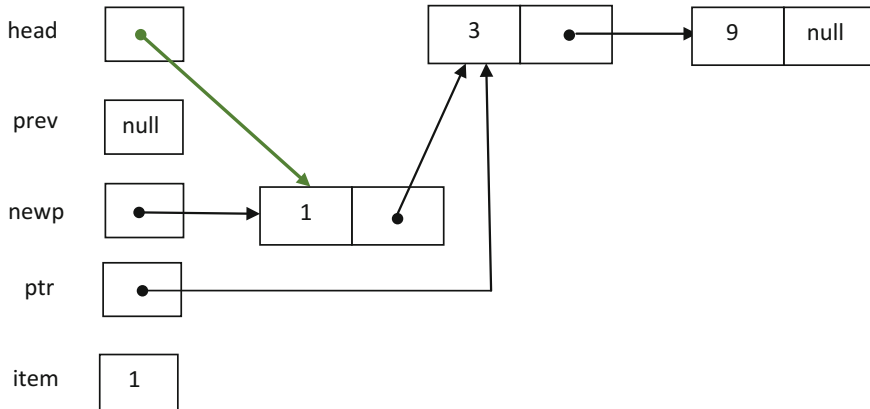
Then the reference in `ptr` is copied to the `next` section of the new node containing the 1 to reference the node containing the 3.



So far all of the code in the segment has worked properly. The problem is that the `prev.setNext(newp)` will cause an exception because `prev` is equal to `null`. How can this problem be solved? Since `prev` is `null`, it is indicating that the current node referenced by `ptr` is the first node in the list. Further, the fact that `prev` is `null` can be used to indicate that the value in `head` should be altered instead of a previous node. This takes the form of an `if` statement as follow:

```
if (prev == null)
    head = newp;
else
    prev.setNext(newp);
```

The results of executing the above code is the following diagram showing that the node containing the number 1 has been added to the beginning of the linked list:

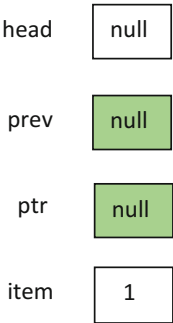


Adding the new `if` statement to the prior code segment results in the following code segment:

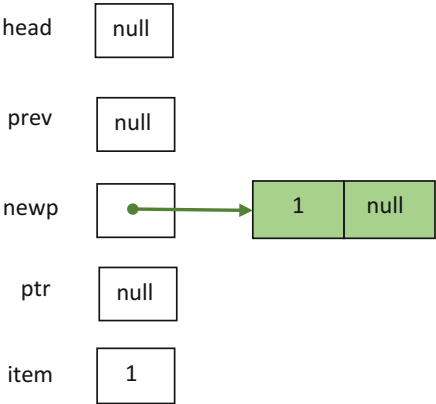
```
ptr = head;
prev = null;
while(ptr != null && ptr.getData() < item) {
    prev = ptr;
    ptr = ptr.getNext();
}
if(ptr == null || ptr.getData() != item) {
    Node newp = new Node();
    newp.setData(item);
    newp.setNext(ptr);
    if(prev == null)
        head = newp;
    else
        prev.setNext(newp);
}
```

6.1.4 Inserting into an Empty List

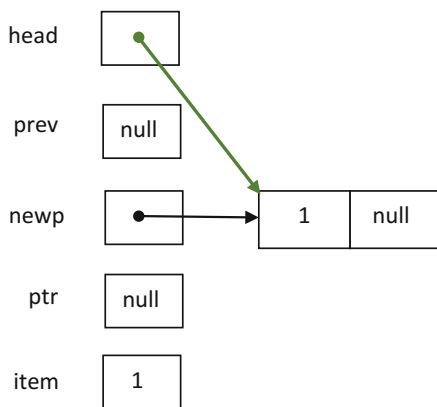
Three out of the four possibilities have been accounted for, so now it is time to consider the case of an empty list. If a list is empty, then `head` would be `null`. Walking through the code segment at the end of the last subsection, `ptr` is initialized to the value in `head` which is `null` and `prev` is initialized to `null` as shown below:



Since `ptr` is equal to `null` the condition in the `while` loop is short circuited, the body of the loop is not executed, and the execution continues on to the `if` statement. In the then section of the `if` statement, since `ptr` is equal to `null`, a new node is created containing the 1 as shown below:



Although the next section of the new node already contains a `null` by the constructor, the `null` of `ptr` is copied into the new node indicating it is the end of the list and since `ptr == null`, the reference in `newp` is copied into `head` indicating it is the beginning of the list as shown below:



This is an interesting feature of creating the first node in a linked list in that it is both the first and last node in the list and the code created for both these scenarios is utilized. Further, note that there were no changes needed to the existing code to make this scenario work. Again, it is helpful to not start writing code for the first node in the list, because often the code previously written for the other scenarios works without many or any alterations.

6.1.5 The Insert Method

Before completing the `insert` method, it is important to check to ensure that all four scenarios work with the algorithm before implementing it. The reason is that when code is added to accommodate one of the new scenarios, it is possible that those changes might introduce new errors to the previous existing scenarios. Once one is sure that no new errors were introduced, the only thing left to add to the algorithm is a `boolean` flag called `inserted` to indicate to the invoking program whether the item has indeed been inserted into the list or not. Also, the declaration of local variables needs to be included along with the method heading that includes the parameter and a `return` statement at the end. The completed `insert` method is as shown in Fig. 6.2.

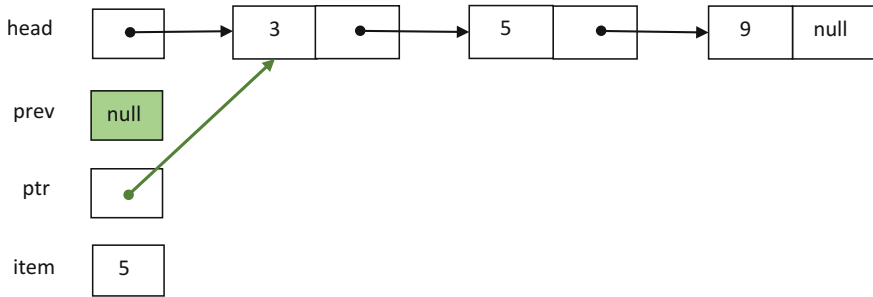

```
public boolean insert(int item) {
    boolean inserted;
    Node ptr, prev;
    inserted = false;
    ptr = head;
    prev = null;
    while(ptr != null && ptr.getData() < item) {
        prev = ptr;
        ptr = ptr.getNext();
    }
    if (ptr == null || ptr.getData() != item) {
        inserted = true;
        Node newp = new Node();
        newp.setData(item);
        newp.setNext(ptr);
        if(prev == null)
            head = newp;
        else
            prev.setNext(newp);
    }
    return inserted;
}
```

Fig. 6.2 insert method

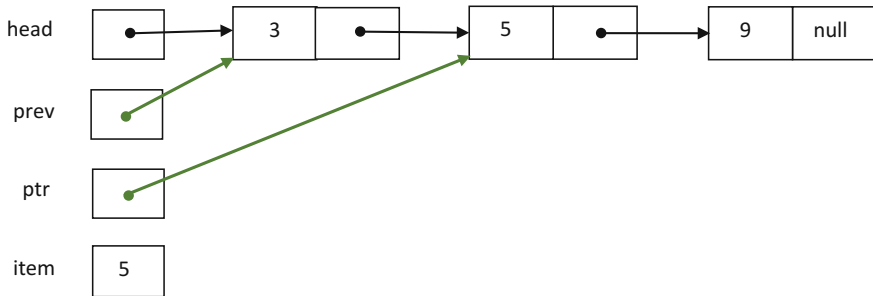
6.2 Creating the Delete Method

Having created the complete insert method in the previous section, it is now time to examine what is needed to create the delete method. However, instead of actually developing the code for the delete method, this section will look at what is needed to create the method. One can then use the same steps outlined in the previous section to write the necessary code as an exercise. Of course there will be differences between the two segments since they perform different tasks, but the techniques involved are the same.

As always, it helps to draw pictures to visualize what needs to be done and also as suggested in the previous section, it helps to start with the generic case before proceeding to the more specialized cases. Again, assuming that the list already exists and that the node containing the number 5 needs to be deleted, the list will need to be searched. Initially ptr will point to the same node as head, and prev will be pointing to null, as shown below:

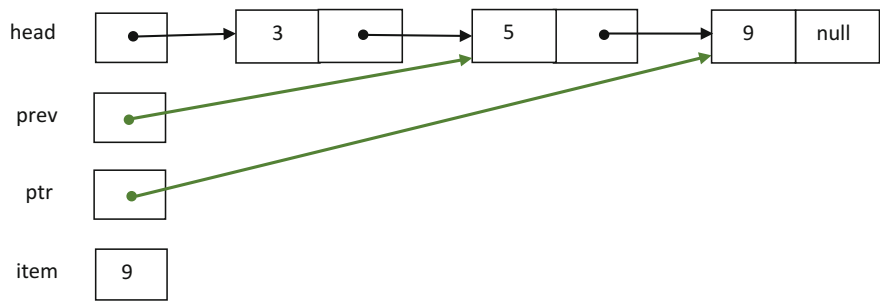


Once the node is found, `ptr` should be referencing the node to be deleted, and `prev` is referencing the node prior to the node to be deleted. Whereas the `insert` method would not insert a number that was already in the list, the `delete` method must be careful not to delete a number that is not in the list. As before, if the node is not found, the loop should not iterate to the end of the list but rather stop where the number would have been located. Also as with the `insert` method, one must make sure that conditional statements short circuit properly.

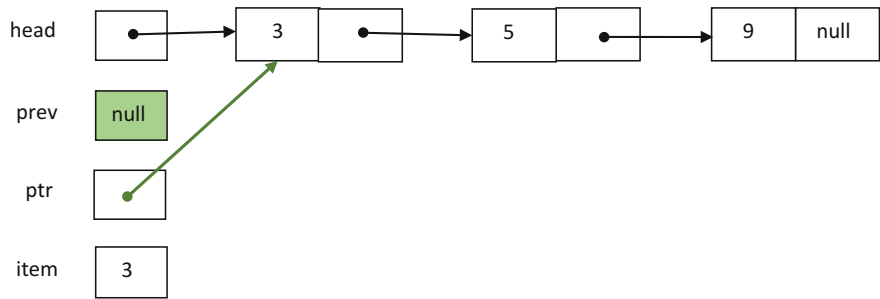


When writing the code for the actual deletion of the node, an `if` statement is probably needed to insure that `ptr` contains a reference and that the node containing the actual number to be deleted exists. Also, be sure to follow the heuristics given in the last section, such as the shortest route is often the best one and that it is usually better to create new links before destroying old links. Further, when there are two routes, make sure that the more general route is followed that can be adapted to other situations, such as using `prev` instead of `head` when trying to delete in the middle of a short list.

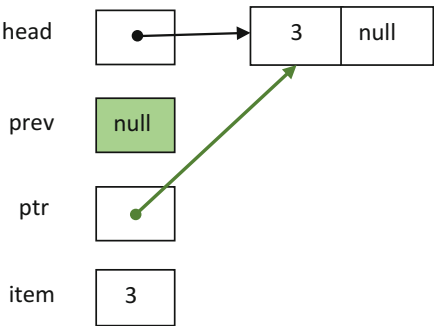
As before, it is a good idea to then work on deleting from the end of the list so as to defer having to deal with `head`. In the following figure it is the 9 that should be deleted:



Only after the middle and end have been taken care of is it a good idea to consider removing the first node at the beginning of a list:



And then to consider a list with only one item in it as follows:



As before, after adding code for the subsequent possibilities, it is important to go back and ensure that any subsequent additions and changes to the code have not altered the functionality of the previously written code. Lastly, the item to be deleted needs to be sent to the method as a parameter, and it is helpful to return a `boolean` value (with a default value of `false`) indicating whether the item has actually been deleted as shown in the following skeleton:

```
public boolean delete(int item) {  
    boolean deleted = false;  
    Node ptr, prev;  
  
    //Body of method  
  
    return deleted;  
}
```

6.3 The `LinkedList` Class

Using the `Node` class from Fig. 5.1, and creating a constructor, including the `insert` method from Fig. 6.2, the `delete` method skeleton from the previous section, the recursive print method from Fig. 5.5 renamed `printR`, along with a print driver called `printRecursive` below, results in the `LinkedList` class as shown in Fig. 6.3.

6.4 Test Program

Once the `LinkedList` class and the associated `Node` class are ready, they need to be tested. One wants to make sure that the `insert` and `delete` methods work flawlessly. This means that a node can be inserted in the middle, end, beginning, and into an empty list. The same goes for deleting in the middle, end, beginning, or from a list with only one node. Of course if there are not to be duplicates, then if an item is already in the list it should not be inserted, and of course if an item is not in the list, it should not be deleted. The following test program prompts for and inputs a command to either insert, delete, print, and stop using a sentinel control loop. For insertion or deletion, the program prompts the user for the number to be inserted or deleted, respectively. If the item is successfully inserted, not inserted, deleted, or not deleted, the appropriate message is output. The print command outputs the contents of the list to the screen, and a stop command causes the program to stop.

```
public class LinkedList {

    private Node head;

    public LinkedList() {
        head = null;
    }

    public boolean insert(int item) {
        boolean inserted;
        Node ptr, prev;
        inserted = false;
        ptr = head;
        prev = null;
        while(ptr != null && ptr.getData() < item) {
            prev = ptr;
            ptr = ptr.getNext();
        }
        if(ptr == null || ptr.getData() != item) {
            inserted = true;
            Node newp = new Node();
            newp.setData(item);
            newp.setNext(ptr);
            if(prev == null)
                head = newp;
            else
                prev.setNext(newp);
        }
        return inserted;
    }

    public boolean delete(int item) {
        boolean deleted=false;
        Node ptr, prev;
        // Body of method
        return deleted;
    }

    public void printRecursive() {
        System.out.print("List Recursive: ");
        printR(head);
        System.out.println();
    }

    private void printR(Node p) {
        if(p != null) {
            System.out.print(p.getData()+" ");
            printR(p.getNext());
        }
    }
}
```

Fig. 6.3 LinkedList Class

```
import java.util.*;
class Ch5 {
    public static void main(String[] args) {

        Scanner scanner;
        scanner = new Scanner(System.in);

        String response;
        int number;

        LinkedList list;
        list = new LinkedList();

        System.out.println();
        System.out.print("Do you wish to i = insert, d = delete, ");
        System.out.print("p = print, or s = stop: ");
        response = scanner.next();
        while(!response.equals("s")) {
            if(response.equals("i")) {
                System.out.print("Enter a number to insert into the list: ");
                number = scanner.nextInt();
                System.out.print("The number " + number);
                if (list.insert(number))
                    System.out.println(" was inserted");
                else
                    System.out.println(" was not inserted");
            }
            else
                if(response.equals("d")) {
                    System.out.print("Enter a number to delete from the list: ");
                    number = scanner.nextInt();
                    System.out.print("The number " + number);
                    if(list.delete(number))
                        System.out.println(" was removed");
                    else
                        System.out.println(" was not removed");
                }
            }
            else
                if(response.equals("p"))
                    list.printRecursive();
                else {
                    System.out.print("The command entered is not i, d, p, ");
                    System.out.println("or s, please try again.");
                }
        }
        System.out.println();
    }
}
```